

Witryny i Aplikacje Internetowe

Autor: Mateusz Dziejczak

Spis treści

1. Instrukcje sterujące w JavaScript.....	2
1.1 Warunki (if, else if, else, switch).....	2
1.2 Pętle (for, while, do...while, for...of, for...in).....	3
1.2.1 Pętla for.....	4
1.2.2 Pętla while.....	5
1.2.3 Pętla do ... while.....	5
1.2.4 Pętla for...in.....	6
1.2.5 Pętla for...of.....	7
1.2.6 Break i Continue w pętlach.....	7
1.2.7 Podsumowanie instrukcji sterujących.....	8
2. Tablice w JavaScript.....	9
2.1 Tworzenie tablicy.....	9
2.2 Indeksy w tablicy.....	9
2.3 Tablica może zawierać różne typy danych.....	10
2.4 Długość tablicy.....	10
2.5 Operacje na tablicach w JavaScript.....	10
2.5.1 Dodawanie elementów do tablicy.....	10
2.5.2 Usuwanie elementów z tablicy.....	11
2.5.3 Modyfikowanie elementów tablicy.....	11
2.5.4 Iteracja po tablicy (przechodzenie przez elementy).....	12
2.5.5 Łączenie tablic (metoda <i>concat()</i>).....	12
2.5.6 Sprawdzanie, czy coś jest tablicą (metoda <i>Array.isArray()</i>).....	12
2.5.7 Tablice wielowymiarowe.....	13
2.5.8 Podsumowanie.....	13
3. Funkcje w JavaScript.....	14
3.1. Czym jest funkcja?.....	14
3.2. Jak tworzyć funkcje w JavaScript?.....	14
3.2.1 Deklaracja funkcji (function declaration).....	14
3.2.2 Funkcje z parametrami.....	15
3.2.3 Funkcje zwracające wartość (return).....	15
3.3 Rodzaje funkcji w JavaScript.....	15
3.3.1 Funkcja anonimowa (function expression).....	15

3.3.2 Funkcja strzałkowa (arrow function) – ES6+.....	16
3.4 Funkcje jako argumenty innych funkcji (Callback).....	16
3.5 Domyślne wartości parametrów.....	17

1. Instrukcje sterujące w JavaScript

Instrukcje sterujące w języku JavaScript pozwalają na kontrolowanie przebiegu wykonywania programu w zależności od określonych warunków lub powtarzających się czynności. Dzięki tym mechanizmom programy stają się bardziej dynamiczne i elastyczne.

- **Instrukcje warunkowe** (np. `if`, `else if`, `else`, `switch`) pozwalają na wykonywanie różnych bloków kodu w zależności od tego, czy dany warunek jest spełniony.
- **Pętle** (np. `for`, `while`, `do...while`) umożliwiają wielokrotne wykonywanie tego samego fragmentu kodu, co jest szczególnie przydatne w sytuacjach, gdy nie znamy z góry liczby operacji.
- **Instrukcje skoku** (np. `break`, `continue`, `return`) pozwalają na zmianę przepływu wykonania programu w obrębie pętli lub funkcji, umożliwiając bardziej zaawansowane manipulowanie logiką programu.

Dzięki tym narzędziom możesz w pełni kontrolować działanie swojego programu, tworząc dynamiczne i responsywne aplikacje.

1.1 Warunki (`if`, `else if`, `else`, `switch`)

Instrukcja `if` pozwala sprawdzić, czy coś jest prawdą, a następnie wykonać odpowiednią akcję. Możemy to porównać do pytania: „Czy to jest prawda?”. Jeśli odpowiedź brzmi „tak”, wykonujemy pewną czynność, jeśli „nie” – robimy coś innego.

```
let age = 20;

if (age < 18) {
    console.log("Jesteś niepełnoletni.");
} else if (age === 18) {
    console.log("Właśnie osiągnąłeś pełnoletność.");
} else {
    console.log("Jesteś pełnoletni.");
}
```

Czasem mamy do czynienia z sytuacjami, gdzie jest więcej niż jeden przypadek, a my chcemy wybrać odpowiednią akcję. W takim przypadku używamy `switch`. W tym przykładzie komputer sprawdza, jaki mamy dzień (np. 2). Jeśli to wtorek, wyświetli „Wtorek”. Przełącznik `default`, wykona się w momencie, kiedy podana opcja nie znajduje się w zakresie przewidzianych przez nas odpowiedzi. Przykładowo, jeżeli użytkownik poda `let day = 4`; wtedy wykona nam się to co zawiera się po słowie `default`, z racji, iż wartość zmiennej równa 4, nie została przez programistę przewidziana. Przy instrukcji `switch`, wymagane jest także dodanie słowa `break`; po każdym przypadku (`case`). Służy ono do tego, aby program wiedział gdzie ma zakończyć przeszukiwanie i wyskoczyć z instrukcji. Bez słowa `case` program nie wiedziałby gdzie ma się zatrzymać i w przypadku wpisania np. `day = 1` bez instrukcji `break`, wykonałby **WSZYSTKIE** instrukcje, aż do `default`, włącznie z nim. Jedynie w przypadku `default` możemy nie pisać słówka `break`, gdyż po tej instrukcji i tak nie ma już innych, dlatego tutaj nie grozi nam dalsze mimowolne wykonanie kodu.

```
let day = 2;

switch(day) {
  case 1:
    console.log("Poniedziałek");
    break;
  case 2:
    console.log("Wtorek");
    break;
  default:
    console.log("Nieznany dzień");
}
```

1.2 Pętle (for, while, do...while, for...of, for...in)

Pętle w JavaScript są jednym z najważniejszych narzędzi, które pozwala na wielokrotne wykonywanie tego samego kodu bez konieczności jego powtarzania. W praktyce, pętle przydają się wszędzie tam, gdzie musimy wykonać jakąś operację wiele razy, np. przejść po elementach tablicy, sprawdzić wiele warunków, czy zrealizować jakąś czynność przez określoną liczbę razy.

1.2.1 Pętla for

Pętla for jest jedną z najczęściej używanych pętli w JavaScript. Jest idealna, gdy dokładnie wiemy, ile razy chcemy powtórzyć jakąś operację. Składa się z trzech części:

```
for (inicjalizacja; warunek; inkrementacja) {  
    // kod do wykonania  
}
```

- **Inicjalizacja:** To miejsce, gdzie ustawiamy początkową wartość zmiennej, która będzie sterować pętlą (np. licznik).
- **Warunek:** Jest to wyrażenie, które sprawdzane jest przed każdą iteracją pętli. Jeśli warunek jest **prawdziwy**, pętla wykona się, w przeciwnym razie zakończy działanie.
- **Inkrementacja (lub dekrementacja):** To wyrażenie, które modyfikuje zmienną sterującą po każdej iteracji. Zwykle jest to zwiększenie lub zmniejszenie licznika.

Przykład:

```
for (let i = 0; i < 5; i++) {  
    console.log(i);  
}
```

- **Inicjalizacja:** `let i = 0` – licznik pętli zaczyna się od 0.
- **Warunek:** `i < 5` – pętla wykona się dopóki zmienna `i` jest mniejsza niż 5.
- **Inkrementacja:** `i++` – po każdej iteracji `i` zostaje zwiększone o 1.

Wynik tego kodu to:

```
0  
1  
2  
3  
4
```

Pętla wykona się 5 razy, zaczynając od 0 i kończąc na 4, ponieważ po każdej iteracji zmienna `i` zostaje zwiększona o 1.

1.2.2 Pętla while

Pętla while jest bardziej ogólna niż for. Jej składnia jest prostsza i opiera się tylko na warunku, który musi być spełniony, aby pętla działała.

```
while (warunek) {  
    // kod do wykonania  
}
```

- **Warunek:** Pętla sprawdza, czy ten warunek jest **prawdziwy**. Jeśli jest, kod wewnątrz pętli będzie się wykonywał.
- Jeśli warunek będzie **fałszywy** już na początku, pętla w ogóle się nie wykona.

Przykład:

```
let i = 0;  
while (i < 5) {  
    console.log(i);  
    i++;  
}
```

W tym przypadku, pętla działa podobnie jak poprzednia for, ale warunek jest oddzielony od samego zwiększania zmiennej i.

Wynik wykonania kodu:

```
0  
1  
2  
3  
4
```

Uwaga: Pętla while jest bardzo użyteczna, gdy nie znamy dokładnej liczby iteracji z góry, ale mamy jakiś warunek, który musi być spełniony.

1.2.3 Pętla do ... while

Pętla do...while jest podobna do pętli while, ale różni się tym, że warunek jest sprawdzany **po** wykonaniu kodu w pętli, a nie przed. Dzięki temu kod w pętli wykona się przynajmniej raz, nawet jeśli warunek jest początkowo fałszywy.

```
do {
```

```
// kod do wykonania  
} while (warunek);
```

Przykład kodu:

```
let i = 0;  
do {  
    console.log(i);  
    i++;  
} while (i < 5);
```

Wynik:

```
0  
1  
2  
3  
4
```

Pętla `do...while` jest przydatna w sytuacjach, gdy chcemy, by kod wykonał się przynajmniej raz, np. w przypadku, gdy użytkownik musi wprowadzić jakąś wartość (np. w formularzu), a następnie chcemy to sprawdzić.

1.2.4 Pętla `for...in`

Pętla `for...in` służy do iteracji po właściwościach obiektów w JavaScript. To oznacza, że możemy używać tej pętli, gdy chcemy przejść po kluczach obiektu, a nie po liczbach.

```
for (let klucz in obiekt) {  
    // kod do wykonania  
}
```

Przykład:

```
let osoba = {  
    imie: "Ania",  
    wiek: 25,  
    kraj: "Polska"  
};  
  
for (let klucz in osoba) {  
    console.log(klucz + ": " + osoba[klucz]);  
}
```

```
}
```

Wynik:

```
imie: Ania
```

```
wiek: 25
```

```
kraj: Polska
```

W tym przypadku pętla `for...in` przechodzi przez wszystkie klucze obiektu `osoba` i wypisuje je na ekranie.

1.2.5 Pętla `for...of`

Pętla `for...of` pozwala iterować po **elementach tablicy** lub innych strukturach danych, takich jak ciągi znaków (stringi), zestawy (Set) czy mapy (Map).

```
for (let element of tablica) {  
    // kod do wykonania  
}
```

Przykład:

```
let owoce = ["jabłko", "banan", "pomarańcza"];  
for (let owoc of owoce) {  
    console.log(owoc);  
}
```

Wynik:

```
jabłko
```

```
banan
```

```
pomarańcza
```

Pętla `for...of` jest bardzo wygodna, gdy chcemy przejść po elementach tablicy i nie interesują nas indeksy tych elementów, tylko ich wartości.

1.2.6 Break i Continue w pętlach

W JavaScript możemy używać instrukcji `break` i `continue` do kontrolowania przebiegu pętli:

- **break:** Zatrzymuje pętlę natychmiastowo, nawet jeśli warunek pętli nie został spełniony.
- **continue:** Pomija resztę kodu w bieżącej iteracji pętli i przechodzi do kolejnej.

Przykład z **break**:

```
for (let i = 0; i < 5; i++) {  
    if (i === 3) {  
        break; // Zatrzymujemy pętlę, gdy i osiągnie 3  
    }  
    console.log(i);  
}
```

Wynik:

```
0  
1  
2
```

W tym przypadku, gdy `i` jest równe 3, instrukcja **break** powoduje, że pętla zatrzymuje się i wykonuje się tylko 3 razy.

Przykład z **continue**:

```
for (let i = 0; i < 5; i++) {  
    if (i === 3) {  
        continue; // Pomijamy resztę kodu, gdy i jest równe 3  
    }  
    console.log(i);  
}
```

Wynik:

```
0  
1  
2  
4
```

W tym przypadku, gdy `i` jest równe 3, instrukcja `continue` powoduje, że **console.log(i)** nie zostanie wywołane dla tego przypadku, ale pętla nadal przejdzie do kolejnej iteracji.

1.2.7 Podsumowanie instrukcji sterujących.

Pętle w JavaScript pozwalają na powtarzanie określonych operacji. Każdy typ pętli ma swoje zastosowanie:

- **for**: Używaj, gdy znasz liczbę iteracji.

- **while:** Używaj, gdy nie wiesz dokładnie, ile razy będzie trzeba wykonać kod, ale masz warunek zatrzymania.
- **do...while:** Używaj, gdy chcesz, aby kod wykonał się przynajmniej raz.
- **for...in:** Używaj, aby iterować po właściwościach obiektów.
- **for...of:** Używaj, aby iterować po elementach tablicy lub innych struktur danych.

2. Tablice w JavaScript

Tablica to struktura, która pozwala przechowywać **więcej niż jedną wartość** w jednej zmiennej. Możesz wyobrazić sobie tablicę jak rząd szufladek w biurku – każda szuflada przechowuje coś innego, ale wszystkie należą do tej samej "szufladki" (czyli tablicy). W tablicach w JavaScript przechowujemy różne rodzaje danych, takie jak liczby, teksty, inne tablice, obiekty itp.

2.1 Tworzenie tablicy

Tablicę tworzymy za pomocą nawiasów kwadratowych `[]`. Możemy od razu umieścić w niej dane, które chcemy przechować. Elementy w tablicy są oddzielane przecinkami.

Przykład:

```
let owoce = ["jabłko", "banan", "pomarańcza"];
```

Tutaj stworzyliśmy tablicę owoce, która zawiera trzy elementy: „jabłko”, „banan” i „pomarańcza”. Możemy nazywać tablicę dowolnie, ale dobrze jest dawać jej nazwy, które będą mówiły, co zawiera.

2.2 Indeksy w tablicy

Każdy element tablicy ma swój **indeks**, który określa jego pozycję w tablicy. Indeksy zaczynają się od 0 (pierwszy element ma indeks 0, drugi ma indeks 1 itd.). Możemy odwołać się do poszczególnych elementów tablicy, używając ich indeksów.

Przykład:

```
let owoce = ["jabłko", "banan", "pomarańcza"];
```

```
console.log(owoce[0]); // jabłko
```

```
console.log(owoce[1]); // banan
```

```
console.log(owoce[2]); // pomarańcza
```

Tutaj w tablicy owoce:

- `owoce[0]` zwróci "jabłko",
- `owoce[1]` zwróci "banan",
- `owoce[2]` zwróci "pomarańcza".

2.3 Tablica może zawierać różne typy danych

Tablica w JavaScript nie musi zawierać tylko jednego typu danych. Może przechowywać liczby, teksty, inne tablice, obiekty, a nawet funkcje!

Przykład:

```
let mieszanka = [23, "jabłko", true, [1, 2, 3], { imie: "Ania", wiek: 25 }];
```

W tablicy mieszanka mamy:

- liczbę 23,
- tekst "jabłko",
- wartość logiczną true,
- inną tablicę [1, 2, 3],
- obiekt { imie: "Ania", wiek: 25 }.

2.4 Długość tablicy

Tablica w JavaScript ma właściwość `length`, która zwraca liczbę elementów w tablicy.

Przykład:

```
let owoce = ["jabłko", "banan", "pomarańcza"];
console.log(owoce.length); // 3
```

W tym przykładzie `owoce.length` zwróci 3, ponieważ tablica `owoce` zawiera trzy elementy.

2.5 Operacje na tablicach w JavaScript

Teraz omówię najczęściej używane operacje na tablicach, które pozwalają na manipulowanie ich zawartością.

2.5.1 Dodawanie elementów do tablicy

Możemy dodawać elementy do tablicy na różne sposoby. Dwa najbardziej popularne to:

- **push()**: Dodaje element na **koniec** tablicy.
- **unshift()**: Dodaje element na **początek** tablicy.

Przykład z **push()**:

```
let owoce = ["jabłko", "banan"];  
owoce.push("pomarańcza");  
console.log(owoce); // ["jabłko", "banan", "pomarańcza"]
```

Przykład z **unshift()**:

```
let owoce = ["banan", "pomarańcza"];  
owoce.unshift("jabłko");  
console.log(owoce); // ["jabłko", "banan", "pomarańcza"]
```

2.5.2 Usuwanie elementów z tablicy

Podobnie jak dodawanie elementów, możemy również usuwać elementy z tablicy:

- **pop()**: Usuwa ostatni element z tablicy.
- **shift()**: Usuwa pierwszy element z tablicy.

Przykład z **pop()**:

```
let owoce = ["jabłko", "banan", "pomarańcza"];  
owoce.pop();  
console.log(owoce); // ["jabłko", "banan"]
```

Przykład z **shift()**:

```
let owoce = ["jabłko", "banan", "pomarańcza"];  
owoce.shift();  
console.log(owoce); // ["banan", "pomarańcza"]
```

2.5.3 Modyfikowanie elementów tablicy

Możemy modyfikować elementy tablicy, przypisując nową wartość do konkretnego indeksu.

Przykład:

```
let owoce = ["jabłko", "banan", "pomarańcza"];  
owoce[1] = "gruszka";  
console.log(owoce); // ["jabłko", "gruszka", "pomarańcza"]
```

Tutaj zmodyfikowaliśmy element o indeksie 1 (czyli "banan") na "gruszka".

2.5.4 Iteracja po tablicy (przechodzenie przez elementy)

Często chcemy przejść przez wszystkie elementy w tablicy i wykonać jakąś operację na każdym z nich. Możemy to zrobić na kilka sposobów:

- **Pętla for:** Tradycyjna pętla, którą znamy z wcześniejszego opisu.
- **forEach():** Metoda tablicy, która wykonuje określoną funkcję na każdym elemencie tablicy.

```
let owoce = ["jabłko", "banan", "pomarańcza"];
for (let i = 0; i < owoce.length; i++) {
    console.log(owoce[i]);
}

let owoce = ["jabłko", "banan", "pomarańcza"];
owoce.forEach(function(owoc) {
    console.log(owoc);
});
```

Oba powyższe przykłady wyświetlą:

```
jabłko
banan
pomarańcza
```

2.5.5 Łączenie tablic (metoda *concat()*)

Czasami chcemy połączyć dwie tablice w jedną. Możemy to zrobić za pomocą metody `concat()`.

Przykład:

```
let owoce1 = ["jabłko", "banan"];
let owoce2 = ["pomarańcza", "gruszka"];
let wszystkieOwoce = owoce1.concat(owoce2);
console.log(wszystkieOwoce); // ["jabłko", "banan", "pomarańcza", "gruszka"]
```

2.5.6 Sprawdzanie, czy coś jest tablicą (metoda `Array.isArray()`)

Warto czasami sprawdzić, czy dana zmienna jest tablicą. Używamy do tego metody `Array.isArray()`.

Przykład:

```
let owoce = ["jabłko", "banan"];
console.log(Array.isArray(owoce)); // true

let liczba = 5;
console.log(Array.isArray(liczba)); // false
```

2.5.7 Tablice wielowymiarowe

Tablice mogą zawierać inne tablice, tworząc **tablice wielowymiarowe**. To przydatne w przypadku, gdy chcemy przechowywać dane w formie macierzy.

Przykład tablicy 2D (dwuwymiarowej):

```
let macierz = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];

console.log(macierz[0][1]); // 2
```

Tutaj mamy tablicę `macierz`, która zawiera trzy inne tablice (w każdym wierszu trzy elementy). Aby uzyskać dostęp do konkretnego elementu, musimy podać dwa indeksy – pierwszy wskazuje wiersz, a drugi kolumnę.

2.5.7 Podsumowanie:

- Tablica to sposób przechowywania wielu elementów w jednej zmiennej.
- Elementy w tablicy są indeksowane od 0.
- Tablica może zawierać różne typy danych (liczby, teksty, obiekty, inne tablice).
- Możemy dodawać i usuwać elementy z tablicy, np. za pomocą metod `push()`, `pop()`, `shift()`, `unshift()`.
- Możemy modyfikować elementy tablicy, a także przechodzić przez nią przy pomocy pętli (`for`, `forEach`).
- Istnieją także tablice wielowymiarowe, czyli tablice w tablicach.

3. Funkcje w JavaScript

Funkcje w JavaScript to **jeden z najważniejszych elementów tego języka**. Pozwalają na wielokrotne wykonywanie tego samego kodu, zwiększają jego czytelność i ułatwiają zarządzanie dużymi programami. Można je porównać do „przepisów na działanie”, które można wywoływać wiele razy bez konieczności przepisywania tego samego kodu.

3.1. Czym jest funkcja?

Funkcja to fragment kodu, który wykonuje **konkretne zadanie**. Możemy ją stworzyć raz, a następnie wielokrotnie wywoływać w dowolnym miejscu programu.

Porównanie do życia codziennego:

Wyobraź sobie funkcję jako **przepis kulinarny**. Masz instrukcję, jak przygotować naleśniki – wystarczy tylko podać składniki (parametry) i wykonać przepis (wywołać funkcję), aby otrzymać gotowy posiłek (wynik funkcji).

3.2. Jak tworzyć funkcje w JavaScript?

3.2.1 Deklaracja funkcji (function declaration)

Najprostszy sposób na stworzenie funkcji to użycie słowa kluczowego `function`, nazwy funkcji oraz nawiasów `{ }`.

Przykład:

```
function przywitaj() {  
    console.log("Cześć! Jak się masz?");  
}
```

Teraz możemy użyć tej funkcji w dowolnym miejscu w kodzie, wywołując ją po nazwie:

```
przywitaj();
```

W konsoli powinien wyświetlić nam się komunikat:

```
"Cześć! Jak się masz?"
```

3.2.2 Funkcje z parametrami

Funkcja może przyjmować dane wejściowe – czyli **parametry**. Możemy dzięki temu przekazywać do funkcji różne wartości i wykonywać działania na podstawie tych wartości.

Przykład:

```
function przywitajImie(imie) {  
    console.log("Cześć, " + imie + "!");  
}
```

przywitajImie("Ania"); → W konsoli wypisze nam "Cześć, Ania!"

przywitajImie("Tomek"); → W konsoli wypisze nam "Cześć, Tomek!"

Tutaj imie to **parametr** funkcji – oznacza, że możemy podać dowolne imię i funkcja dostosuje swój wynik.

3.2.3 Funkcje zwracające wartość (return)

Czasami chcemy, aby funkcja zwróciła nam konkretną wartość, zamiast tylko wyświetlać coś na ekranie. Do tego używamy słowa kluczowego **return**.

```
function dodaj(a, b) {  
    return a + b;  
}
```

```
let wynik = dodaj(5, 3);
```

console.log(wynik); → W konsoli dostaniemy zawartość zmiennej wynik, czyli to co zwróciła nam funkcja dodaj, w tym przypadku – 8

Tutaj funkcja dodaj () zwraca sumę dwóch liczb, którą możemy później wykorzystać w programie.

3.3 Rodzaje funkcji w JavaScript

JavaScript daje nam kilka różnych sposobów na tworzenie funkcji.

3.3.1 Funkcja anonimowa (function expression)

Funkcja anonimowa to taka, która **nie ma nazwy** i jest przypisywana do zmiennej.

Przykład:

```
const powitanie = function() {
```

```
    console.log("Cześć!");  
};
```

powitanie(); → W konsoli dostaniemy napis "Cześć!".

3.3.2 Funkcja strzałkowa (arrow function) – ES6+

Wprowadzone w nowoczesnym JavaScript tzw. **funkcje strzałkowe** (=>) pozwalają na jeszcze krótszy zapis.

Przykład:

```
const powitanie = () => {  
    console.log("Cześć!");  
};
```

powitanie(); → W konsoli dostaniemy napis "Cześć!"

Jeśli funkcja ma **tylko jedną instrukcję**, można ją jeszcze bardziej skrócić.

Przykład:

```
const dodaj = (a, b) => a + b;  
console.log(dodaj(2, 3)); → W konsoli dostaniemy wynik operacji dodawania - 5
```

3.4 Funkcje jako argumenty innych funkcji (Callback)

Funkcje w JavaScript są tzw. **"first-class citizens"**, co oznacza, że można je traktować jak zmienne – przekazywać jako argumenty do innych funkcji.

Przykład:

```
function wykonaj(funkcja) {  
    funkcja();  
}  
  
wykonaj(() => console.log("To jest callback!"));
```

Przykładem takiego podejścia jest metoda setTimeout(), która wykonuje podaną funkcję po określonym czasie.

Przykład:

```
setTimeout(() => {
```

```
    console.log("Minęły 2 sekundy!");  
  }, 2000);
```

3.5 Domyślne wartości parametrów

Możemy ustawić domyślną wartość parametru, jeśli użytkownik nie poda żadnej wartości.

Przykład:

```
function powitanie(imie = "Gościu") {  
    console.log("Cześć, " + imie + "!");  
}  
  
powitanie(); // "Cześć, Gościu!"  
powitanie("Kasia"); // "Cześć, Kasia!"
```

3.6 Funkcje rekurencyjne (rekurencja)

Funkcja może wywoływać samą siebie – nazywa się to **rekurencją**. Jest to przydatne np. w obliczaniu silni (n!).

Przykład:

```
function silnia(n) {  
    if (n === 0) {  
        return 1;  
    }  
    return n * silnia(n - 1);  
}  
  
console.log(silnia(5)); // 120
```

3.7 Funkcje wbudowane w JavaScript

JavaScript oferuje wiele gotowych funkcji, które możemy używać bez definiowania ich samemu:

- `Math.random()` – generuje losową liczbę
- `Math.floor()` – zaokrągla liczbę w dół
- `parseInt()` – zamienia tekst na liczbę
- `setTimeout()` – opóźnia wykonanie kodu
- `setInterval()` – powtarza kod w określonych odstępach czasu

Przykład:

```
console.log(Math.random()); // np. 0.7382827  
console.log(Math.floor(4.9)); // 4  
console.log(parseInt("42")); // 42
```

3.8 Podsumowanie

1. **Funkcja** to blok kodu, który można wielokrotnie wykonywać.
2. Można tworzyć **funkcje nazwane, anonimowe, strzałkowe i callbacki**.
3. Funkcja może mieć **parametry**, które pozwalają na przekazywanie wartości.
4. Możemy zwracać wartość za pomocą **return**.
5. Istnieją funkcje **rekurencyjne, domyślne parametry**, a także funkcje **wbudowane** w JavaScript.